

概率论与数理统计编程作业

杨礼谦 2022080054 机械 24

我选择使用 R 编程语言完成这次的编程作业。

1. 向量操作 (1 分)

1.1 产生一个等差数列: 1, 5, 9, ..., 101

答: 直接调用 R 语言的 sequence 函数 - seq()

以下是 1 到 101, d=4 的等差数列:

```
> seq(1,101,4)
[1] 1 5 9 13 17 21 25 29 33 37 41 45 49 53 57 61 65 69 73 77 81 85
[23] 89 93 97 101
```

1.2 产生一个长度为 8 的字符型向量 "tsinghua", 再把向量的第六个元素变成大写字母。

答: 首先, 调用 R 语言的 split string 函数 strsplit () 以按照每个字符拆分

"tsinghua" 字符串并生成一个字符型向量 char_vector。

接着, 调用 toupper 函数将字符型向量中的第六个元素变成大写字母。

```
> char_vector <- strsplit("tsinghua", NULL)[[1]]
> char_vector[6] <- toupper(char_vector[6])
> char_vector
[1] "t" "s" "i" "n" "g" "H" "u" "a"
```

2. 矩阵操作 (1 分)

2.1 产生矩阵 x: $\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$ 和 y: $\begin{pmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{pmatrix}$

调用 R 语言自带的矩阵函数 - matrix() 生成矩阵 x 和 y

1:9 表示 1 到 9, 3 表示 3 行矩阵, 列由 R 通过数据和行数自行填充

byrow = TRUE 意思是将 1:9 序列按行填充

```
> x <- matrix(1:9,3,byrow = TRUE)
> x
  [,1] [,2] [,3]
[1,]  1   2   3
[2,]  4   5   6
[3,]  7   8   9
> y <- matrix(9:1,3,byrow= TRUE)
> y
  [,1] [,2] [,3]
[1,]  9   8   7
[2,]  6   5   4
[3,]  3   2   1
```

2.2 按照矩阵乘法：计算 x 和 y 的乘积
利用 R 语言中执行矩阵乘法操作符 `%*%`

```
> x%*%y
      [,1] [,2] [,3]
[1,]    30    24    18
[2,]    84    69    54
[3,]   138   114    90
```

2.3 将矩阵 x 的第三行和 y 的第三行互换，输出新的两个矩阵

创造新的矩阵行 - `new_row`

将 x 的第三行， y 的第三行和 `new_row` 相互交叉赋值就能达到互换的效果

```
> new_row <- x[3,]
> x[3,] <- y[3,]
> y[3,] <- new_row
> x
      [,1] [,2] [,3]
[1,]     1     2     3
[2,]     4     5     6
[3,]     3     2     1
> y
      [,1] [,2] [,3]
[1,]     9     8     7
[2,]     6     5     4
[3,]     7     8     9
```

3. 常用统计量 (1 分)

3.1 产生 20 个服从区间 $[0, 1]$ 上均匀分布的随机数

调用 `runif()` 函数生成服从均匀分布的随机数

3.2 计算这些随机数的样本均值、方差

调用 `mean()` 和 `var()` 函数

3.3 计算这些随机数的中位数，下四分位数

调用 `median()` 和 `quantile()` 函数

```
> r_nums <- runif(20, min=0, max=1)
> r_nums
 [1] 0.35677931 0.40712348 0.50131121 0.33229543 0.49541158 0.75854983 0.07546901 0.75560354
 [9] 0.84569578 0.29721214 0.45314999 0.06380633 0.03558580 0.43377434 0.80138121 0.12256897
[17] 0.12383918 0.65147974 0.46869537 0.43057812
> mean(r_nums)
[1] 0.4205155
> var(r_nums)
[1] 0.06421157
> median(r_nums)
[1] 0.4321762
> quantile(r_nums,0.25)
      25%
0.2538689
```

4. 正态随机数生成 (1 分)

4.1 生成 100 个标准正态分布随机数

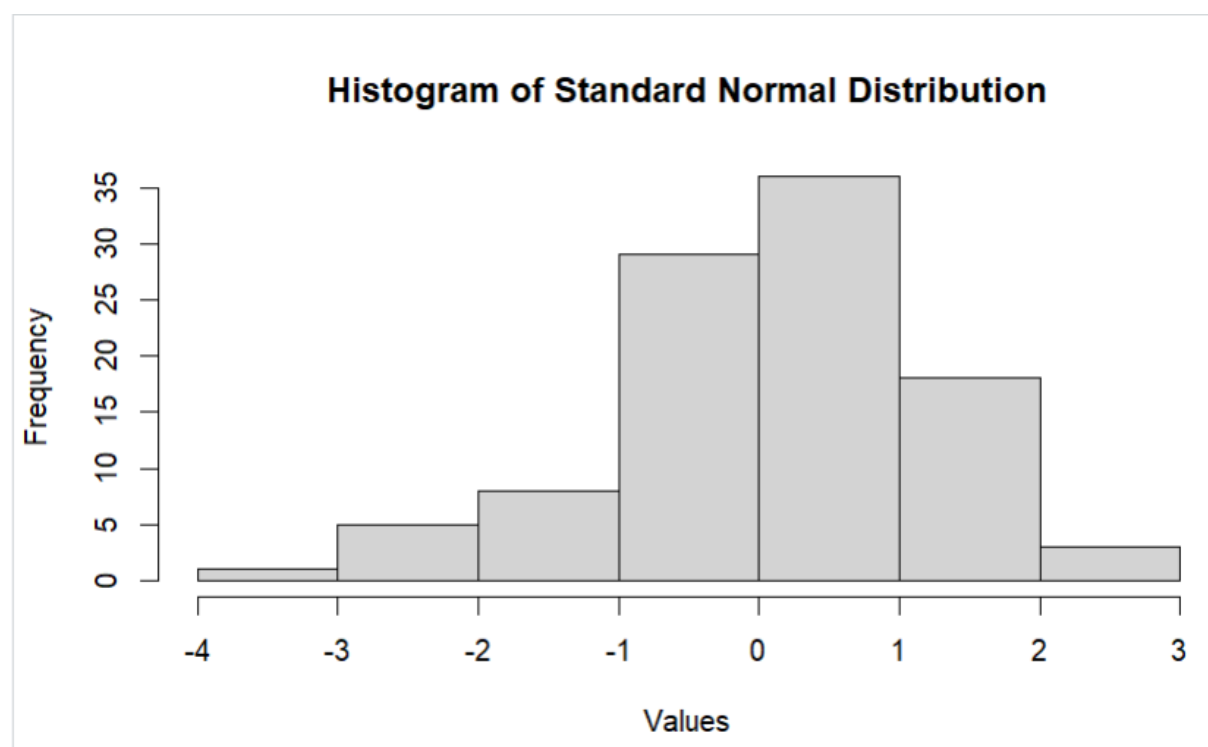
调用 `rnorm()` 函数生成服从标准正态分布的随机数

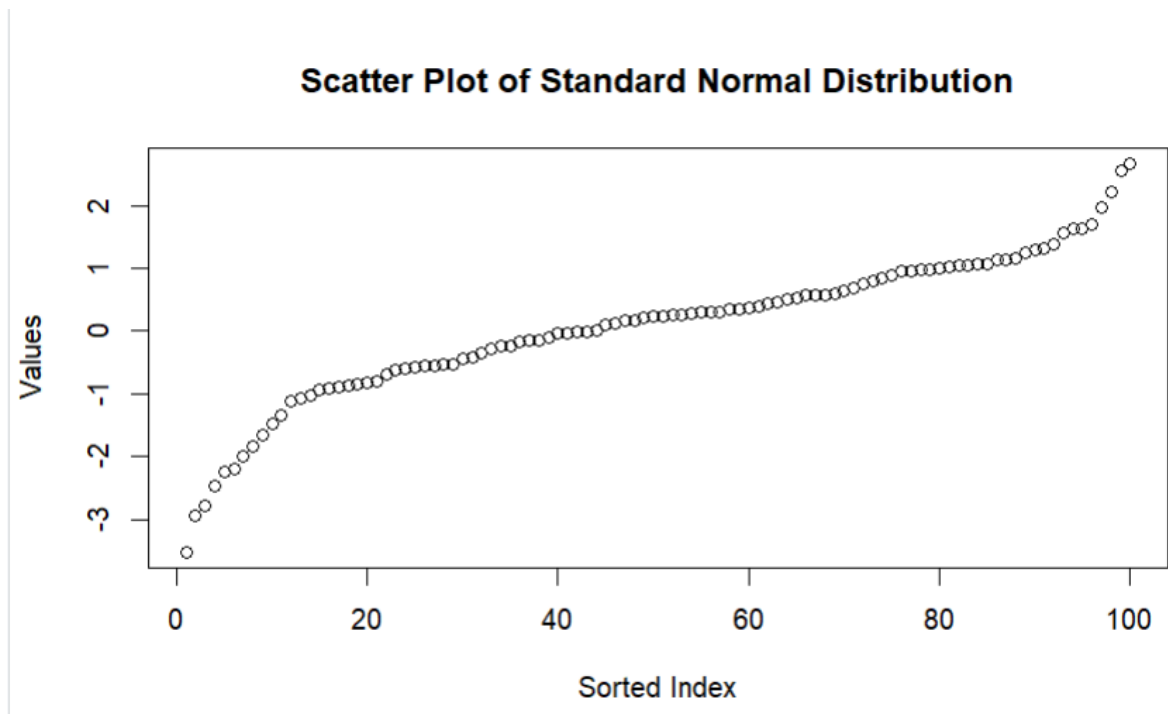
```
> r_nums1 <- rnorm(100,0,1)
> r_nums1
 [1] -2.249013129  0.982954082 -0.231052946  0.950160174 -0.150360421  1.283740434
 [7] -0.896429648 -1.840836207  1.311154989  0.521959975  1.068898963 -0.789118646
[13] -0.583875976 -0.929019265  1.384512262 -0.148440009 -2.929621726 -0.546183516
[19] -0.426415943 -0.679336354 -1.030390371  1.011550441  1.143923087 -1.483198455
[25]  0.511471658  0.175705692  1.629043238  2.667742776 -0.831386202 -0.616677051
[31] -0.874374874 -0.900703890  0.295824156  0.209489737  1.971837165 -1.104661599
[37] -0.355323265 -0.096239700  0.804386596  0.112691704  1.036919973  0.248198246
[43]  0.569574185 -1.330708998 -1.990961774  0.244569739  0.347224645 -3.523554951
[49] -0.010510360  0.897640792  0.749218942 -0.844992194  0.349091584 -0.537508329
[55]  1.237231260  1.704760398  0.694021686  1.071853545  1.025078957 -0.039876690
[61]  0.643297685  0.257004830 -0.042548034  0.305984420  0.453556928  0.598260270
[67] -1.071059115 -0.004229132 -0.159212445  1.150916177  0.239583786 -2.455598175
[73]  0.843594313 -0.544262903  1.637310122  0.565634592  0.018030768  0.371448798
[79]  0.973380589  0.953073300 -2.183649608 -1.663933134 -0.524159196  0.571757883
[85]  2.555822683 -2.771460207 -0.433004797  1.552068054  0.172061204  0.275954563
[91]  1.129114794  1.042759096  0.301377817  2.226280960  0.109015635  0.399925019
[97] -0.281411738 -0.605875912  0.439579629 -0.240213824
```

4.2 画出这些随机数的直方图和散点图 (散点图中横坐标是随机数从小到大排序后的序号)

调用 `hist()` 函数绘制随机数的直方图

调用 `plot(sort())` 函数绘制排序后的随机数向量的散点图



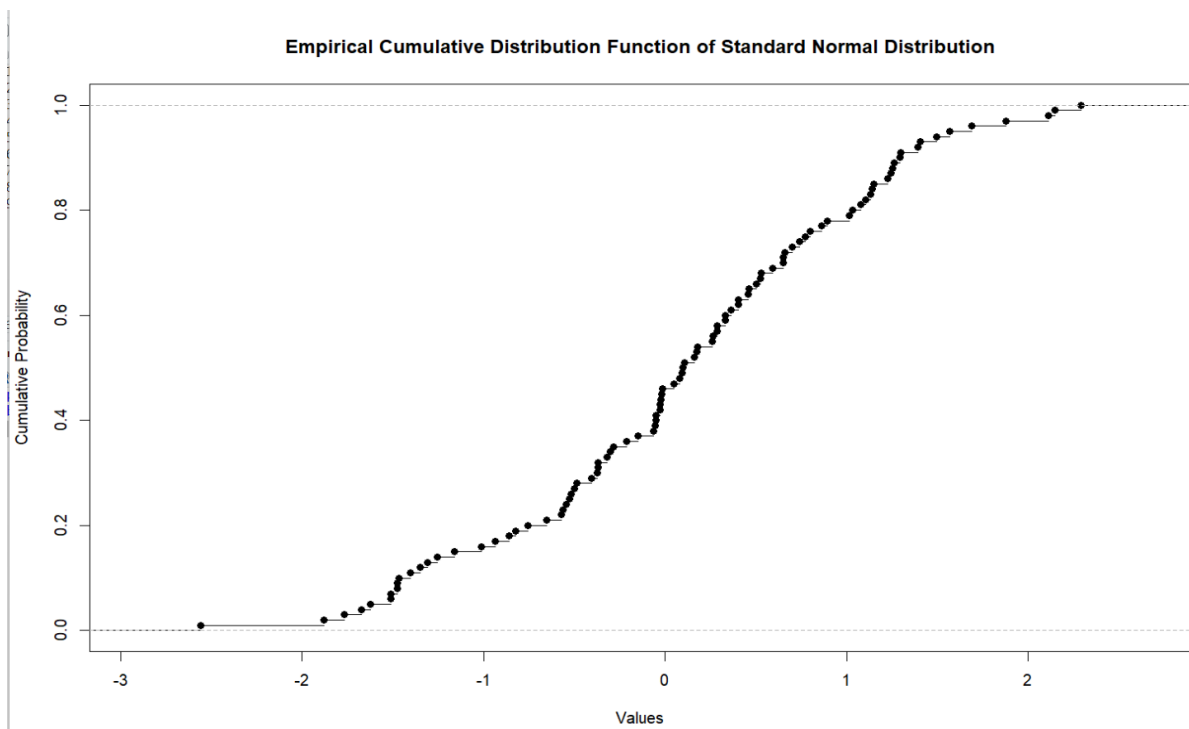


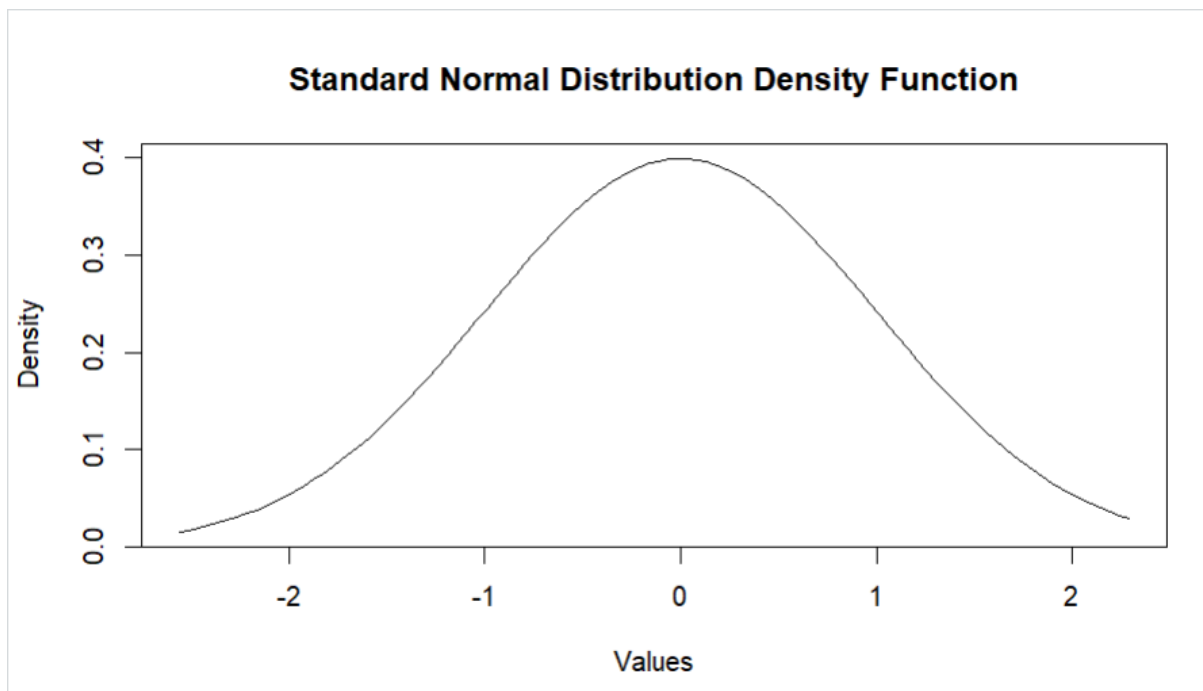
4.3 画出这些随机数的经验分布函数，以及标准正态分布的密度函数

i) 调用 `plot(ecdf())` 函数绘制经验分布函数

ii) 标准正态分布的密度函数分为两部分 x 和 y 轴

- x 轴的值就是由 100 个随机数组成的数列
 - y 轴的值是随机数的概率密度值, 调用 `dnorm()` 函数可求得该处的概率密度值
- 最后调用 `plot()` 函数绘制出由 x - y 轴组成的标准正态分布的密度函数。





5. 正态分布相关计算 (1 分)

写出一个计算程序，任意输入一个(0,1)区间内的概率值，程序可以输出对应的标准正态分布对应于该概率值的分位数。

i)调用 `readline(prompt=“ ”)`函数读取用户输入的概率值，通过 `as.numeric()`函数将其转化为数值好让我们调用 `qnorm()`函数。

ii)为了确保用户输入的概率值是有效的，我写了个简单的判断语句，若输入值不属于[0,1]区间则程序终止。

iii)最后调用 `qnorm()`函数得到对应的标准正态分布对应于该概率值的分位数。

```
> probability <- as.numeric(readline(prompt = ""))
0.75
> if (is.na(probability) || probability < 0 || probability > 1) { stop }
> quantile_value <- qnorm(probability)
> quantile_value
[1] 0.6744898
```